

AutoML and AutoRAG Strategies

Version 1, 2026-09-15

Home

Goal

Teach technical professionals how to strategically evaluate use cases and apply AutoML (predictive modeling) and AutoRAG (generative retrieval) in Red Hat OpenShift AI 3.5+ to drive measurable business value by showcasing its targeted automated AI capabilities.

Objectives

On completing this course, you should be able to:

- **Predictive Foundations (AutoML):** Evaluate when to apply AutoML to convert raw tabular data or time series into production-ready predictive models across classification, regression, and time-series tasks.
- **AutoML governance & deployment:** Use AutoML evaluation artifacts (leaderboards, feature importance, ROC/PR curves) to validate business logic and promote winning models to production.
- **GenAI prompting & MLflow governance:** Transition from predictive data to generative text, leveraging the GenAI Playground and MLflow to version, track, and eliminate "prompt drift".
- **Generative retrieval (AutoRAG):** Position AutoRAG as an automated optimization engine that benchmarks chunking, embedding, and vector database configurations for grounded LLM applications.
- **Knowledge review and assessment:** Track core enablement concepts across AutoML, MLflow prompt tracking, and AutoRAG to validate practitioner capability in diagnosing enterprise data scenarios and selecting optimal AI patterns.

Audience

- **Primary Personas:** Data Scientists, AI Engineers, and Technical Solution Architects evaluating or adopting Red Hat OpenShift AI 3.5+.
- **Technical Level & Focus (TL2):** Technical practitioners and field specialists looking to understand the "why", decision points, trade-offs, and business outcomes of automated capabilities. Focused on transitioning from manual ML/RAG development without needing to perform cluster admin setup procedures.
- **Distribution & Accessibility:** Publicly discoverable enablement designed for global Red Hat field teams, ecosystem partners, and enterprise customers.

Prerequisites

This course assumes that you have the following experience:

- Familiarity with core machine learning concepts (classification, regression, time series).

- Familiarity with generative AI concepts (LLMs, prompting, retrieval-augmented generation).
- Familiarity with Red Hat OpenShift AI 3.5+ from a practitioner (Data Scientist / AI Engineer) perspective.

Notice that no cluster administration or infrastructure setup experience is assumed or required.

Classroom environment

This course requires no hands-on user labs. It uses diagrams, Summit Arcade interactive clickthroughs, and video walkthroughs to demonstrate AutoML and AutoRAG capabilities.

Other sources of information about Red Hat OpenShift AI

For documentation about Red Hat OpenShift AI, see either its [product documentation](#) or the [Red Hat AI Examples](#) upstream repositories.

Author

Karlos Knox

Content & Learning Lead

Red Hat - Product & Technical Learning Team



Use of Artificial Intelligence: This publication contains content or media that is generated or assisted by AI tools. All the Training deliverables have primarily human authorship, and all content was reviewed by a human domain expert.

Predictive Foundations with AutoML

Goal

Evaluate when to apply AutoML to convert raw tabular data or time series into production-ready predictive models across classification, regression, and time-series tasks.

Here's a story you've probably lived through, or at least watched from the next desk over.

A team gets handed a business problem, predict which customers are about to churn, flag the fraudulent transactions, forecast next quarter's demand. Someone spins up a notebook. Weeks go by cleaning the data. More weeks testing algorithms, tuning hyperparameters, arguing about which metric matters. Three, maybe six months later there's a model that works beautifully... on a laptop. And then it quietly dies there, because nobody ever built the path from "notebook that works" to "endpoint that serves." We even have a name for it: **PoC death**.

AutoML in Red Hat OpenShift AI 3.5+ is the answer to that story. It takes the raw tabular data you already have sitting in S3, automates the tedious middle, data prep, feature engineering, algorithm selection, ensembling, and hands you back a ranked leaderboard of production-ready candidate models. Not in months. In hours.

This chapter is about the **foundations**: why this matters to the business, where predictive AI fits alongside the LLMs everyone's talking about, how to tell whether your data is even a good fit, and how to match any business problem to the right kind of prediction. By the end you won't be running models yet, that comes later, but you'll know exactly when AutoML is the right tool to reach for, and why.

Why AutoML? The Business Case

Estimated reading time: 8 minutes.

Objective

Explain, in language a business leader will care about, what AutoML actually automates and the return it delivers.

Let's start with the part that gets budget approved, because if you can't explain the "why" to the person signing the checks, the coolest model in the world never ships.

The problem AutoML is solving

Traditional machine learning projects have a nasty habit of taking three to six months and then stalling out. The data scientist does heroic work, cleaning messy columns, inventing features, testing XGBoost against a neural net against something they read about last week, tuning knobs until the accuracy nudges up another half a percent. It's skilled, careful, **slow** work. And a huge amount of it is the same work, every single time, on every single project.

That repetition is exactly what AutoML automates. Instead of a person hand-running each step, the platform handles:

- **Data preparation**, inferring column types, imputing missing values, encoding categories.
- **Feature engineering**, turning raw columns into signals a model can learn from.
- **Algorithm selection**, trying many model families in parallel instead of one at a time.
- **Ensembling**, combining the best candidates into something stronger than any single model.

You still bring the judgment, the business question, the data, the decision about what "good" looks like. AutoML takes over the grind in the middle.

The "Sorting Hat"

If you want a mental picture, think of AutoML as a **Sorting Hat**. You feed in raw tabular CSV data pulled from S3 storage, the platform reads the patterns hiding in it, and out comes a **ranked leaderboard** of candidate models, sorted best to worst, ready for you to inspect and promote. You're no longer building models one at a time and hoping. You're reviewing a graded shortlist.

The return on investment

Here's how that translates when it's your turn to justify the spend:

- **70–80% faster time-to-market**. Development cycles that used to run three to six months collapse into a matter of hours. Multi-quarter R&D efforts become same-week operational delivery.
- **A 3x workforce multiplier**. You no longer need a scarce, specialized ML PhD for every project. Software engineers, data analysts, and IT practitioners can stand up high-accuracy predictive baselines themselves, which frees your specialists for the genuinely hard problems.
- **No more PoC death**. This is the quiet one that matters most. Because AutoML produces **standardized deployment artifacts**, the winning model moves directly toward a production endpoint instead of getting stranded in a notebook. The path from experiment to production is built in, not bolted on later.

What's next

Faster and cheaper is a great pitch, but only if you're pointing it at the right kind of problem. Next we'll place predictive AI next to the large language models everyone's talking about, so you can see exactly where each one earns its keep, and learn how to tell whether your data is even a candidate for AutoML in the first place.

Where Predictive AI Fits (and Where It Doesn't)

Estimated reading time: 10 minutes.

Objective

Position predictive models against LLMs on cost and behavior, and qualify whether a dataset is a good fit for AutoML.

Walk into almost any room in 2026 and the conversation is about large language models. Which is great, LLMs are genuinely transformational. But there's a trap hiding in all that excitement:

reaching for a giant GPU-hungry model to answer a question that a tiny, boring, deterministic model would answer better, cheaper, and a thousand times faster.

So before we go further, let's get honest about where predictive AI actually shines.

Predictive models vs. LLMs

The easiest way to see it is side by side. Same job description, two very different employees:

Dimension	Predictive Models (AutoML / Tabular)	Large Language Models (LLMs)
Primary scope	Narrow, single-task structured decisions (churn, fraud, pricing)	Broad, open-domain reasoning and text generation
Input structure	Low-dimensional structured tabular data (10–100+ features)	High-dimensional unstructured text across billions of tokens
Artifact size	Kilobytes to megabytes (lightweight trees / ensembles)	Tens to hundreds of gigabytes (billions of parameters)
Hardware	Lightweight CPU infrastructure, no specialized GPUs	High-VRAM GPU clusters
Latency	Microsecond inference for high-throughput systems	Second-level response times
Auditability	Deterministic logic with clear feature importance	Probabilistic reasoning needing guardrail evaluation

The takeaway is worth saying plainly: **predictive models are cheap, fast, and deterministic**. They scrape millions of rows on the CPU clusters you already own, at a fraction of the cost of a GPU fleet, and they give you the same answer every time with a clear paper trail for **why**. When the question is "will this transaction be fraud?" you don't need a model that can also write poetry. You need one that's right, fast, and explainable.

Is your data even a candidate?

AutoML is powerful, but it's not magic, it expects a certain shape of data. Before you kick off a run, check that your dataset qualifies:

- **Tabular structure.** Rows are entities, columns are attributes, stored as CSV. Files should be UTF-8, comma-delimited, with a header row. You can upload up to 32 MiB through the dashboard UI, or ingest up to 100 MB directly from S3.
- **A clear target label.** One column holds the ground-truth answer you want to predict for new data.
- **Feature diversity.** A healthy mix of numeric and categorical columns, that's where AutoML's automatic type inference, missing-value imputation, and encoding earn their keep.
- **Enough volume.** At least 500–1,000 rows to prototype, and 10,000+ is typical for a serious enterprise evaluation.

When AutoML is not the tool

Just as important is knowing when to put it down and reach for something else:

- **Unstructured media**, images, audio, video. Those need specialized deep-learning pipelines.
- **Open-ended text generation**, chatbots, document synthesis, summarization. That's the world of LLMs and AutoRAG, which we'll get to later in the course.
- **Custom domain edge cases**, problems that hinge on hand-crafted business rules a model can't infer from the raw table.

Match the tool to the shape of the problem and AutoML feels like a superpower. Force it onto the wrong problem and it just frustrates everyone. Now that you can spot a good fit, let's look at the four flavors of prediction it can actually do.

The Four Prediction Task Types

Estimated reading time: 10 minutes.

Objective

Match any predictive business problem to one of the four AutoML task types and its default optimization metric.

Here's the good news: no matter how complicated the business problem sounds, every predictive question you'll ever hand to AutoML boils down to one of just **four** task types. Learn to spot which one you're looking at and you're most of the way there.

Even better, you often don't have to decide alone. When you load a CSV into the Red Hat OpenShift AI dashboard, it inspects your columns, infers their types, and **recommends** a task type and a sensible default metric. Think of it as the Sorting Hat whispering a suggestion, but you should understand the four houses well enough to know when it's right.

Start by reading your data, not the problem statement

If this is your first time driving one of these tools, here's the habit to build early: don't start from how the business **describes** the problem, start from what your data actually looks like. The stakeholder might say "I want to know who's going to leave us," but the platform doesn't see intentions, it sees columns. So the very first thing you do is open your CSV and look hard at **one** column: the **target**, the value you want the model to predict for new rows.

Two quick questions answer almost everything:

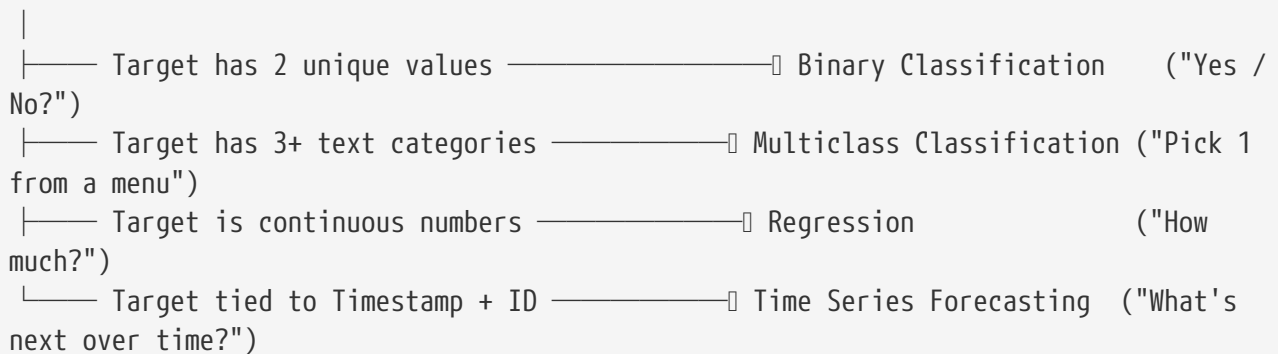
1. **What kind of value lives in the target column?** Is it a handful of labels, or a number that can slide up and down a scale?
2. **Is time part of the story?** Is each row anchored to a date or timestamp, with something identifying which sequence it belongs to?

Answer those and the task type falls out on its own. That's the skill this section builds, so that when the dashboard recommends a task type, you can confirm it rather than take it on faith.

The quick decision tree

The clue is almost always in your **target column**, the thing you're trying to predict:

Raw CSV Data



Let's take them one at a time.

1. Binary classification: "Yes or No?"

The workhorse of enterprise ML. Your target column has exactly **two** values, 0/1, True/False, Churn/No-Churn.

- **Data cue:** two unique values in the target.
- **You'll see it in:** customer churn prevention, fraud detection, credit-risk approval.
- **Default metric:** **Accuracy**, but switch to **ROC-AUC** or **F1** when the classes are lopsided. On a dataset that's 99% legitimate and 1% fraud, a model that guesses "not fraud" every time is 99% accurate and completely useless, so accuracy alone will lie to you.

2. Multiclass classification: "Pick 1 from a menu"

Same idea as binary, but now there are three or more categories to choose from.

- **Data cue:** three or more distinct text labels in the target.
- **You'll see it in:** support-ticket routing, customer segmentation, classifying **which type** of equipment failure occurred.
- **Default metric:** **Accuracy**.

3. Regression: "How much?"

When the answer isn't a category at all but a number on a continuous scale.

- **Data cue:** the target holds continuous numeric values, with no time component.
- **You'll see it in:** price estimation, predicting a patient's medical charges, estimating how long an operation will take.
- **Default metric:** **R² (R-squared)**, the share of variance the model explains, on a 0.0-to-1.0 scale. Closer to 1.0 is better.

4. Time series forecasting: "What's next over time?"

Regression's time-aware cousin. You're not just predicting a number, you're predicting it **forward**, respecting the order of events.

- **Data cue:** the dataset has a **timestamp** column, an **ID** column identifying each sequence (say, each store or product), and a numeric **target**.
- **You'll see it in:** retail inventory demand, quarterly sales projections, capacity planning.
- **Default metric: MASE (Mean Absolute Scaled Error)**, it grades the forecast against a naive "tomorrow looks like today" baseline, so you can tell whether the model is genuinely adding value or just going through the motions.

What's next

You can now hear a business problem, glance at the target column, and name the task type and the metric that matters, the core reflex of a predictive practitioner. In the next chapter we'll open the hood: how AutoML actually builds and stacks these models under the covers, and how to read the evaluation "report card" before you promote a winner into production.

Validating and Deploying AutoML Models

Goal

Use AutoML evaluation artifacts (leaderboards, feature importance, ROC/PR curves) to validate business logic and promote winning models to production.

In chapter 1 you learned to walk up to a business problem, read your data, and pick the right task type and metric. So let's pick up the story right where a first run leaves you: the pipeline has finished, and you're looking at a leaderboard full of models you didn't hand-build.

The tempting move is to grab whatever sits in the top row and ship it. Resist that for a few more minutes. A high score tells you a model fit the **training** data well, it doesn't yet tell you the model is measuring the **right thing**, or that it will behave in production. This chapter is about earning the confidence to promote a model on purpose rather than on faith.

We'll do it in three moves: first a quick look under the hood at what AutoML was actually doing while you waited, then how to read the full evaluation "report card" that comes back with every run, and finally the paved path from a winning model to a live, serving endpoint. By the end you'll be able to validate a predictive model and put it into production without needing a platform administrator to hold your hand.

Under the Hood: How AutoML Builds Its Models

Estimated reading time: 8 minutes.

Objective

Explain what AutoGluon is doing during a run and why the leaderboard is an ensemble, not a single algorithm.

You don't strictly **need** to know what happens between "start run" and "here's your leaderboard" to use AutoML. But a little bit of under-the-hood knowledge pays off the moment you start reading results, because the names you'll see on the leaderboard suddenly mean something.

AutoGluon, the conductor

Under the covers, RHOAI AutoML is powered by **AutoGluon**, which acts as a meta-orchestrator. Instead of asking you to bet on one algorithm and tune it by hand, AutoGluon runs **parallel sweeps** across several model families at once, then figures out which ones actually earned their place. Think of it less as a single musician and more as a conductor auditioning a whole orchestra.

Those candidates come from two broad families.

"Smart flowchart builders" (gradient-boosted decision trees)

These models learn by chaining together lots of yes/no splits, like a flowchart that keeps correcting its own mistakes. They're the bread and butter of tabular prediction:

- **XGBoost**, the meticulous veteran. Uses exact, depth-wise tree splits. A dependable default for

standard spreadsheet and financial data.

- **LightGBM**, the speed demon. Built by Microsoft, it uses fast histogram-based, leaf-wise splitting to chew through multi-gigabyte datasets with low memory overhead.
- **CatBoost**, the category specialist. Built by Yandex, it handles categorical text columns natively, so you don't have to manually one-hot encode them.

"Synthetic brains" (deep neural networks)

- **Tabular MLPs (PyTorch / FastAI)**, multi-layer perceptrons that capture complex, non-linear interactions between your columns, patterns the tree models can miss.

The real trick: stacked ensembling

Here's the part that matters most. AutoGluon doesn't just crown a single winning algorithm and walk away. It combines the predictions of the top candidates into a multi-layer **Weighted Ensemble**, letting the models cover for each other's blind spots. That stacked approach consistently beats hand-tuning any single model by roughly **2–5% in accuracy**.



First time seeing these names? Don't worry about choosing between XGBoost, LightGBM, and the rest, letting you **skip** that decision is the whole point of AutoML. Just know that when a "WeightedEnsemble" model tops your leaderboard, it isn't a bug or a mystery entry: it's usually the blended model, and it's often your best performer.

What's next

Now that you know the leaderboard is a graded audition rather than a single guess, let's actually read it, along with the rest of the report card AutoML hands back with every run.

Reading the Report Card

Estimated reading time: 12 minutes.

Objective

Interpret the leaderboard and evaluation artifacts to validate a model's business logic before promoting it.

When a run finishes, RHOAI gathers everything the pipeline produced and lays it out in the dashboard as a set of connected views. Together they're the model's **report card**. Your job here isn't to admire the top score, it's to ask each artifact a question and make sure the answers hang together before you trust the model with a real decision.

Let's walk them in the order you'll naturally use them.

The leaderboard: who's in the running

The leaderboard is your starting point. It lists every candidate model AutoML trained, ranked by

the primary optimization metric for your task (the Accuracy, R^2 , or MASE you met in chapter 1). But don't stop at the ranking column. It also shows **secondary metrics**, **fit times**, and **inference latencies**, and those extra columns often decide the real winner.



The top model isn't automatically the right one to ship. If the number-one model scores a hair higher but takes ten times longer to return a prediction, a slightly lower-ranked model with microsecond latency may be the smarter production choice. Read across the row, not just down the rank.

Feature importance: is it learning the right thing?

This is the artifact that separates "the model scored well" from "the model is trustworthy." **Feature importance** works by shuffling each column's values and measuring how much the model's performance drops, which tells you how much the model actually leans on that input.

Use it to answer three questions:

- **Does this match reality?** If a churn model's top signals are "months since last login" and "support tickets opened," that lines up with business intuition. Good sign.
- **Is there anything suspicious?** A single feature that dominates everything else is a classic red flag for **data leakage**, where a column accidentally encodes the answer (for example, a "cancellation_date" field sneaking into a churn model). The model looks brilliant in testing and falls apart in production.
- **Can I explain this to a stakeholder?** Feature importance is how you turn a model into a story a business owner will actually believe.



New to this? Make feature importance the **first** thing you look at after the score, not the last. Catching a leaked column here saves you from deploying a model that's secretly cheating.

Confusion matrix: where classification goes wrong

For binary and multiclass models, the **confusion matrix** lays out actual versus predicted classes in a grid: true positives, false positives, false negatives, and true negatives. It moves you past a single accuracy number and shows you **which** mistakes the model makes.

That distinction is often a business decision in disguise. In fraud detection, a false negative (missing real fraud) usually costs far more than a false positive (flagging a legitimate charge for review). The matrix lets you see that trade-off in black and white instead of hoping the headline accuracy has it covered.

ROC and PR curves: tuning the trade-off

The **ROC (Receiver Operating Characteristic)** and **Precision-Recall** curves visualize how the model separates classes across every possible **decision threshold**, not just the default cutoff. This is where you calibrate the model to the business.

Turn the fraud model's sensitivity up and you catch more fraud but annoy more legitimate

customers with reviews; turn it down and the reverse. These curves are the dial, and they let you pick the operating point that matches what your organization actually cares about.

Time series backtesting: does it hold up over time?

For forecasting models there's no single test set, because time only runs one direction. Instead, RHOAI does **rolling backtesting**: it walks the model forward through successive validation windows and charts how the forecast error behaves. You can compare predicted versus observed values in the best and worst windows, which quickly reveals whether a model is reliably useful or just got lucky in one quarter.

Putting it together

None of these views is decisive on its own. A model earns promotion when the story is consistent: a strong leaderboard metric, feature importance that matches reality with no leakage, an error profile you can live with, and, for forecasts, stable behavior across backtests. Read them together and you're validating **business logic**, not just chasing a score.

What's next

You've validated a model you believe in. Now let's move it out of the dashboard and into production, along the paved path from a finished run to a live serving endpoint.

From Winner to Endpoint: The Production Handoff

Estimated reading time: 8 minutes.

Objective

Follow the repeatable path that promotes a validated model from a completed run to a live serving endpoint.

Remember "PoC death" from chapter 1, the model that works beautifully and then never leaves the notebook? This is the section that prevents it. Once you've picked a model you trust, RHOAI gives you a short, repeatable path to get it serving real traffic, and none of it requires cluster-admin skills.

There are three exits from a completed run, and you'll use them for different reasons.

Export the notebook (for the tinkerers)

Click **Save notebook** and AutoML generates a Python Jupyter notebook with the code already written to connect to your S3 storage, load the trained predictor, and run batch scoring or your own custom validation.

Reach for this when you want to poke at the model yourself, score a big batch of records offline, or hand a data scientist a running start. It's the "show me the code" door.

Register the model (for governance)

Register promotes the winning model version straight into the **RHOAI Model Registry**. Think of the registry as the system of record for your models: it versions them, keeps their metadata, and gives your team one trusted place to answer "which model is this, and where did it come from?"



First time shipping a model? Register it even if you're eager to deploy. The registry is what makes a model **governable**, versioned, discoverable, and auditable, rather than a mystery file someone deployed one afternoon.

Deploy to a serving endpoint (for production)

This is the one that turns a file into a service. RHOAI deploys the model using the **AutoGluon ServingRuntime for KServe**, which instantly stands up a scalable REST API. Your application then gets predictions by POSTing to a standard endpoint:

```
/v2/models/<model-name>/infer
```

That's a **KServe v2** inference endpoint. Any service that can make an HTTP request, your web app, a batch job, another microservice, can now get live predictions without knowing a thing about how the model was built.



If terms like KServe and "serving runtime" are new, here's the short version: KServe is the piece of OpenShift AI that runs your model as a web service and scales it for you. The AutoGluon ServingRuntime is a prebuilt engine that already knows how to load and serve AutoML models, so you don't assemble any of that plumbing yourself.

The full arc

Step back and look at the path you've traced across two chapters:

```
Raw CSV (S3) —> AutoML Run —> Leaderboard —> Evaluation Report Card —> Registry —> KServe REST Endpoint
```

Raw data went in one end, and a governed, validated, production-ready prediction service came out the other, in hours rather than months, and without you touching a cluster.

What's next

You can now take predictive, tabular data all the way from a CSV to a served model. In the next chapter we shift from numbers to **text**: an introduction to generative AI in the GenAI Playground, and using MLflow to version and govern prompts the same disciplined way you just governed models.

Prompt Engineering and MLflow Governance

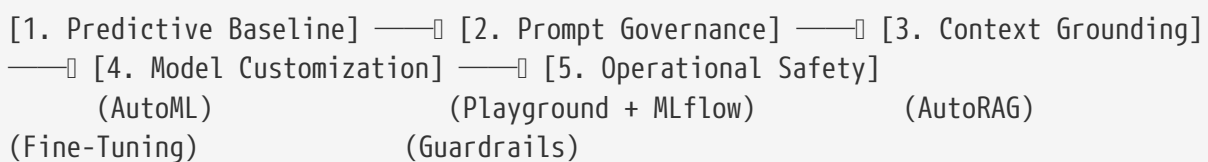
Goal

Transition from predictive data to generative text, leveraging the GenAI Studio Playground and MLflow to version, track, and eliminate "prompt drift".

So far this course has lived in the world of **numbers**: rows, columns, and a target value you wanted to predict. This chapter is where we cross over into **text**. Same platform, same governance instincts you built in chapters 1 and 2, but now the thing you're shaping is a prompt to a large language model instead of a tabular prediction.

Here's the mindset shift worth making up front. When you worked with AutoML, you never would have shipped a model without a leaderboard, feature importance, and a version in the registry. Yet the industry's default habit with prompts is the opposite: a clever instruction gets typed into a text box, copied into some code, tweaked by a teammate, pasted into a notebook, and six weeks later nobody can say which version is actually running in production. That sprawl has a name, **prompt drift**, and it's exactly the kind of chaos this chapter teaches you to prevent.

It helps to see where this fits in the bigger picture. Think of enterprise GenAI as an escalation ladder, and prompt governance is the second rung:



You've already got rung 1. Over the next few pages we'll cover rung 2 in three moves: testing prompts interactively in the **GenAI Studio Playground**, treating those prompts as versioned **code** rather than throwaway text, and using **MLflow** to govern them across your whole organization. Rung 3, AutoRAG, is the very next chapter.

Testing Prompts in the GenAI Studio Playground

Estimated reading time: 10 minutes.

Objective

Use the Playground to test system instructions and generation settings, and understand what each knob actually does.

Before you version a prompt, you have to **find** a prompt worth keeping. That's what the Playground is for: a place to experiment with a language model in real time, watch how it responds, and turn knobs until the behavior is right, all before a single line of it touches your application code.

In the RHOAI dashboard you'll find this under **GenAI Studio**. That menu item has two areas that work together: **Playground**, the interactive testing surface we cover here, and **Prompts**, the

registry where good prompts go to be saved and versioned (that's the next section).

From numbers to text

Quick reframe as you shift gears. With AutoML, you gave the model structured columns and it returned a structured prediction. With a language model, your "input" is **language itself**, the instructions and context you write, and the output is generated text. The quality of that output depends enormously on how you phrase the ask. So the Playground is where prompt **engineering** happens: small changes in wording and settings, big changes in behavior.

The knobs you'll actually turn

The Playground lets you adjust a handful of settings interactively and see the effect immediately. The three you'll reach for most:

- **System instructions** set the model's standing orders: its role, tone, and rules ("You are a concise support agent. Never invent policy numbers."). This is the single biggest lever on behavior.
- **Temperature** controls randomness. Low values keep answers focused and repeatable; higher values make them more varied and creative.
- **Top-p (nucleus sampling)** is a second dial on variety, limiting the model to the most probable words. It's often tuned alongside temperature rather than instead of it.

You'll also keep an eye on the **context window**, the total amount of text (your instructions, the conversation, any supplied content) the model can consider at once. Every model has a limit, and understanding it now will matter a lot in the next chapter when we start feeding models retrieved documents.



First time touching these? Change **one** setting at a time. If you drop the temperature and rewrite the system instruction in the same breath, you won't know which change fixed (or broke) the output. Treat it like tuning an instrument, not flipping every switch at once.

Comparing and tracing

The Playground isn't just a single chat box. Two capabilities make it a genuine evaluation tool:

- **A/B comparison** puts two prompt versions, or two different foundation models, side by side so you can judge outputs directly instead of relying on memory of "what it said last time."
- **Tracing** records the call tree behind a response (using OpenTelemetry under the hood), so you can see what actually happened on the way to an answer, which becomes invaluable as prompts feed into larger agentic workflows.



The models you test against are the foundation models your platform serves (for example Llama, Granite, or Claude). A prompt that shines on one model may behave differently on another, which is exactly why comparing across models is built in.

What's next

Once the Playground gives you a prompt that behaves the way you want, the worst thing you can do is leave it sitting in a browser tab. Next we'll treat that prompt the way you'd treat any other production asset: as versioned code.

Prompts as Code: The MLflow Prompt Registry

Estimated reading time: 11 minutes.

Objective

Understand why prompts deserve version control, and how the MLflow Prompt Registry versions, aliases, and binds them to model settings.

Let's name the problem this section solves. A prompt is not a throwaway string, it's a **parameter** that changes your application's behavior just as surely as any line of code. Yet prompts tend to get treated like sticky notes: copied, pasted, tweaked, and scattered. When a production answer suddenly regresses, "which prompt caused this?" becomes a question nobody can answer. That's prompt drift, and the fix is to give prompts the same discipline you already give models.

That's what the **MLflow Prompt Registry** provides. In the dashboard it lives under **GenAI Studio** → **Prompts**, and if you governed a model in chapter 2, the idea will feel familiar: it's a central place to create, version, tag, and reuse prompt templates across your organization.

Versioning, the Git way

Every time you register a change to a prompt, MLflow saves a new numbered version (v1, v2, v3...) with a commit message describing what changed, just like a Git history for your instructions. Two properties make this trustworthy:

- **Versions are immutable.** Once v2 exists, it never silently changes. If you want to edit a prompt, you create v3. That guarantee is what makes a past result reproducible.
- **You can compare versions side by side**, with the differences highlighted, so you can see exactly how the wording evolved between v1 and v2.

Aliases: pointing at "the good one"

Here's the feature that keeps version numbers from leaking into your application code. An **alias** is a movable label that points at a specific version, for example a **production** alias pointing at v2, or the built-in **latest** alias that always tracks the newest version.

Your application asks for the prompt **by alias**, not by number. So when you promote v3, you just move the **production** alias to v3 and every app picks it up, no code change, no redeploy. Need to undo a bad prompt? Move the alias back. That's a one-click rollback.



New to this idea? Think of an alias like a "current" label on a shelf. The versioned bottles never change, but the label can be moved to whichever one is in service today. Your code always reaches for the label, never a specific bottle.

Prompts carry their settings with them

A prompt is more than its words. The registry can store a **model configuration** alongside each prompt: the model name, temperature, max tokens, and the other generation settings you dialed in back in the Playground. That means the recipe travels with the prompt, so a teammate loading v2 gets not just the text but the exact settings it was tuned for.

Prompts can also be **bound to a specific model provider and endpoint** (Llama, Granite, Claude, and so on). Because the application references the prompt rather than hardcoding a model, teams can swap the underlying model without rewriting prompt code.



Templates use `{{variable}}` placeholders, so a single registered prompt can be reused with different runtime values (a `{{question}}` or a `{{num_sentences}}` filled in at call time). You register the reusable **shape** of the prompt once, not a hundred near-identical copies.

See it in action

The end-to-end registration flow, creating a prompt, committing a v2, comparing versions, and loading one back into the Playground, is best seen rather than read.

A short walkthrough demo accompanies this section and steps through registering and versioning a prompt in the UI.

What's next

You can now version a single prompt like code. The last piece is scale: how these registries are isolated per project, shared across the enterprise, and tied into a complete audit trail. That's governance, and it's next.

Governance at Scale: Workspaces, Lineage, and Audit

Estimated reading time: 9 minutes.

Objective

Explain how MLflow workspaces isolate and share prompts, and why the resulting audit lineage matters to the enterprise.

Versioning a single prompt is a personal win. Governance is the **team** win: making sure the right people can reach the right prompts, that nothing important is ever untracked, and that when someone in risk or compliance asks "what generated this answer six months ago?", you have a real answer.

Workspaces: one per project

MLflow organizes work into **workspaces**, and RHOAI maps them cleanly to something you already know: an OpenShift project namespace corresponds **1:1** to an MLflow workspace. Your project's experiments, runs, and registered prompts all live inside that boundary, and access is controlled by

the same Kubernetes RBAC that governs the rest of the platform.



If "namespace" and "RBAC" are new terms, the short version is: a namespace is a walled-off area for one project's work, and RBAC is the rulebook for who's allowed in. You don't set this up yourself as a practitioner, you inherit the isolation your project already has.

Global registries: shared, vetted prompts

Isolation is great for day-to-day work, but some prompts deserve to be shared across the whole company, think a security-vetted system instruction that every team should build on. For that, a cluster administrator can designate certain namespaces as **global** prompt registries, granting other teams either read-only or edit access.

The result is a central, trusted library of enterprise prompts: individual teams iterate in their own workspaces, while the organization maintains a shared shelf of approved instructions nobody has to reinvent.

Complete audit lineage

This is the payoff that makes governance more than bookkeeping. For every inference, MLflow can log the exact prompt string, the input variables, the model ID, the generation settings, any evaluation scores, and the output that was produced. Nothing floats free.

That lineage buys you three things practitioners and their organizations genuinely need:

- **Reproducibility.** You can recreate any past result because you know precisely what produced it.
- **Rollback.** When a production response regresses, you trace it to a specific prompt version and revert, exactly the alias move from the last section, now with the evidence to justify it.
- **Compliance.** Risk and audit teams can prove which prompt, model, and parameters generated any historical response, which is often a hard requirement in regulated industries.

One platform for models and prompts

Here's a detail that's easy to undervalue: this is the **same** MLflow that tracked your AutoML accuracy scores in chapter 2. The instance versioning your predictive models also versions your prompt templates and traces your LLM calls. Your team doesn't juggle one tool for predictive ML and a separate one for GenAI, it's one governed platform across both, which is a big part of why the discipline actually sticks.

Where this leaves you

Step back and look at what you can now do. You can take raw tabular data all the way to a served predictive model (chapters 1 and 2), and you can take a raw idea for a prompt all the way to a versioned, governed, auditable instruction bound to a model (this chapter). Both halves of "connecting data to models" are now under proper control.

But a well-governed prompt still only knows what the model already knows. The moment you need it grounded in **your** proprietary documents, prompt engineering hits a ceiling. That's rung 3 of the ladder, and it's where the next chapter picks up: **AutoRAG**, which automates the retrieval pipeline that feeds your governed prompts the right context.

Optimizing Retrieval with AutoRAG

Goal

Position AutoRAG as an automated optimization engine that benchmarks chunking, embedding, and vector database configurations for grounded LLM applications.



Not started yet.

This chapter shows how AutoRAG grid-searches the RAG configuration space and how the diagnostic metric triad tells you whether a retrieval pipeline is production-ready.

Benchmarking the RAG Search Space

Estimated reading time: 15 minutes.

Objective

Prepare evaluation input for AutoRAG and interpret the diagnostic metric triad that scores each retrieval configuration.

▶ [autorag-demo.mp4](#) (video)

AutoRAG Demo - OpenShift AI 3.5



Not started yet.

Input preparation

AutoRAG is driven by an evaluation JSON dataset. Each test case provides a `question`, `correct_answers` (multiple valid phrasings), and `correct_answer_document_ids` that identify the grounding sources.

Automated search space and optimization

AutoRAG grid-searches the configuration space that practitioners normally tune by hand:

- Chunking strategy — recursive vs. hybrid structural
- Search mode — vector vs. hybrid
- Fusion strategy — RRF or weighted
- Embedding model — for example, BGE-M3, with multilingual auto-detection

The diagnostic metric triad

Evaluate each run against three complementary metrics:

- **Answer Faithfulness** — a hallucination check.
- **Answer Correctness** — a ground-truth match.

- **Context Correctness** — retrieval accuracy.

Production handoff and inspection

Debug pipelines visually with the Pipeline DAG, then export Responses API code snippets (cURL, Python, Go, Node.js) or indexing and inference Jupyter notebooks for handoff.

What's Next

You have now seen every capability in the course individually. The final chapter synthesizes AutoML, MLflow, and AutoRAG into one end-to-end workflow and validates your understanding with a graded assessment.

Knowledge Review and Assessment

Goal

Track core enablement concepts across AutoML, MLflow prompt tracking, and AutoRAG to validate practitioner capability in diagnosing enterprise data scenarios and selecting optimal AI patterns.



Not started yet.

This chapter connects every capability in the course into one end-to-end workflow, then validates your understanding with a scenario-based graded assessment.

Connecting Data to Models: The End-to-End Workflow

Estimated reading time: 12 minutes.

Objective

Combine AutoML, MLflow, and AutoRAG into a single decision workflow and reason about when automation alone is sufficient.



Not started yet.

End-to-end workflow synergy

The capabilities in this course combine into a live business workflow: AutoML risk scores plus AutoRAG-generated credit memos power a loan officer's decision loop, showing predictive and generative AI working together.

Diagnostic decision logic

Metric combinations tell a diagnostic story. For example, High Faithfulness combined with Low Correctness signals that the retrieval pipeline is missing document context rather than hallucinating.

When "Auto" is enough

Establish clear decision gates for staying with automated capabilities versus escalating to SLM fine-tuning (the subject of Course 2). Knowing where "Auto" stops is as important as knowing what it can do.

What's Next

With the full workflow and decision logic in view, the final section is a scenario-based graded assessment that validates your ability to diagnose enterprise data scenarios and select the optimal AI pattern.

Quiz: Diagnosing Scenarios and Selecting AI Patterns

Estimated reading time: 15 minutes.

Objective

Validate practitioner capability by diagnosing enterprise data scenarios and selecting the optimal AI pattern across AutoML, MLflow, and AutoRAG.

Interactive Assessment

```
<iframe type="text/javascript" src='_attachments/quiz.html' style="width: 1024px; height: 800px" allowfullscreen webkitallowfullscreen mozAllowFullScreen allow="autoplay *; fullscreen *; encrypted-media *" frameborder="0"></iframe>
```

What's Next

Congratulations — you have completed **Governing AI Data Workflows: AutoML and AutoRAG Strategies**. You can now strategically evaluate use cases and apply AutoML and AutoRAG in Red Hat OpenShift AI 3.5+ to drive measurable business value. This course is the first entry in the "Connecting Data to Models and Agents" series; Course 2 continues into SLM fine-tuning for scenarios where automation alone is not enough.